

NEQSIM / ENGINEERING BOOKS

CONTINUOUS AGENTIC ENGINEERING *with NeqSim*

Goal-Directed Solving, Living Tasks
and Software Improvement

Even Solbraa

2 0 2 6

Preface

This is the third volume in a sequence. Industrial Agentic Engineering with NeqSim explains the physics engine, agents, skills and reproducible studies. Agentic Engineering for Oil and Gas Facility Operations connects engineering evidence, tools and specialists in a facility workflow. This volume follows a task after its first answer: it gives the task a goal, keeps its evidence current, re-opens it when conditions change, and turns gaps found in use into tested improvements to NeqSim.

The examples use invented facilities and synthetic measurements. The diagrams are conceptual illustrations, not process drawings or validation data. A public example demonstrates a software contract; engineering use still requires an appropriate fluid, model, operating basis, constraints and independent review.

Two loops carry the argument. A living engineering task revisits the goal as measurements, assumptions and models change. A software improvement campaign revisits a reproducible gap through issues, tests, review and a new version. They can inform each other, but each has its own evidence and acceptance criteria. The worked public reference case gives a small place to start before connecting an existing study or scheduling repeated work.

Implementation snapshot. The command names, paths and examples in Chapters 4–10 were checked against the public `equinor/neqsim` default branch on 27 September 2026. A source checkout can move faster than a packaged release. Run `neqsim --help` and inspect the linked current guide before using a command in another version. The book distinguishes implemented public features, optional integrations and recommended campaign patterns.

Contents

Preface	1
1 From an answer to a living task	3
2 The architecture and its boundaries	6
3 Goals, stop states and the improvement ledger	10
4 Run the public reference case	12
5 Make an existing study living	14
6 Qualify data, detect change and backtest	17
7 Run, schedule, triage and promote	21
8 The NeqSim model side in detail	23
9 A complete teaching walkthrough	26
10Continuous improvement of NeqSim itself	28
11How the two kinds of schedule connect	32
12State of the art and evidence of progress	34
13Governance and deployment	36
14Practical reference	38

Chapter 1

From an answer to a living task

The first answer to an engineering question is a snapshot. It has a date, evidence, assumptions, model version and reviewer. New measurements arrive; equipment ages; a document is revised; a numerical model changes. An answer that was well supported on Monday need not remain the best answer on Friday. Continuous agentic engineering is the practice of retaining the question and the evidence chain so the work can be repeated and improved when something material changes.

The task is the durable unit of work. It contains the original brief, objective, constraints, model, inputs, observed KPIs, baseline, individual cycles, review decisions and software provenance. An agent can prepare and investigate a change. NeqSim computes the physical predictions. A deterministic runner manages repetition and state. The engineer decides whether an action is appropriate and whether a new result should become the accepted baseline.



Figure 1.1: Conceptual view of a living model. Repeated observations prompt comparison with the physical process; the drawing represents no particular facility.

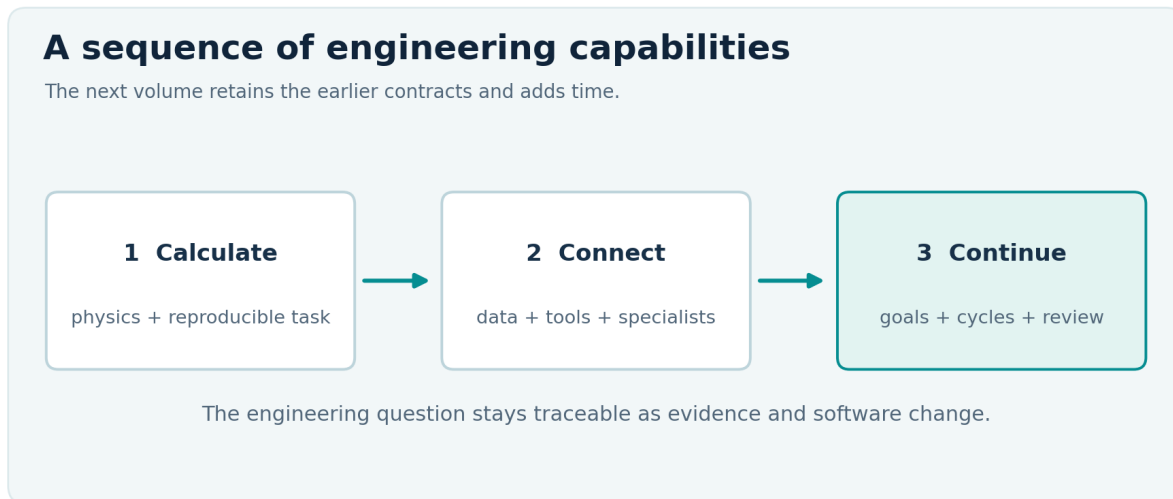


Figure 1.2: The trilogy’s progression. The third volume preserves the first two volumes’ calculation and evidence contracts while adding time, goals and feedback.

1.1 Two loops with different clocks

The **solve loop** tries candidate actions until it reaches a validated target, gains become marginal, an attainable bound rules out the target, a person must decide, or the work budget is exhausted. The **monitor loop** runs periodically or on fresh evidence. It checks source health, recomputes KPIs, compares against a promoted baseline and raises triggers. When a goal regresses or new evidence changes the question, monitoring reopens solving.

The loops must not be confused. A scheduled monitoring cycle is not a request to change a plant every day. A solver’s best candidate is not an operating instruction. A model calibration can reduce a residual without increasing physical production. Each has a different outcome and acceptance test.

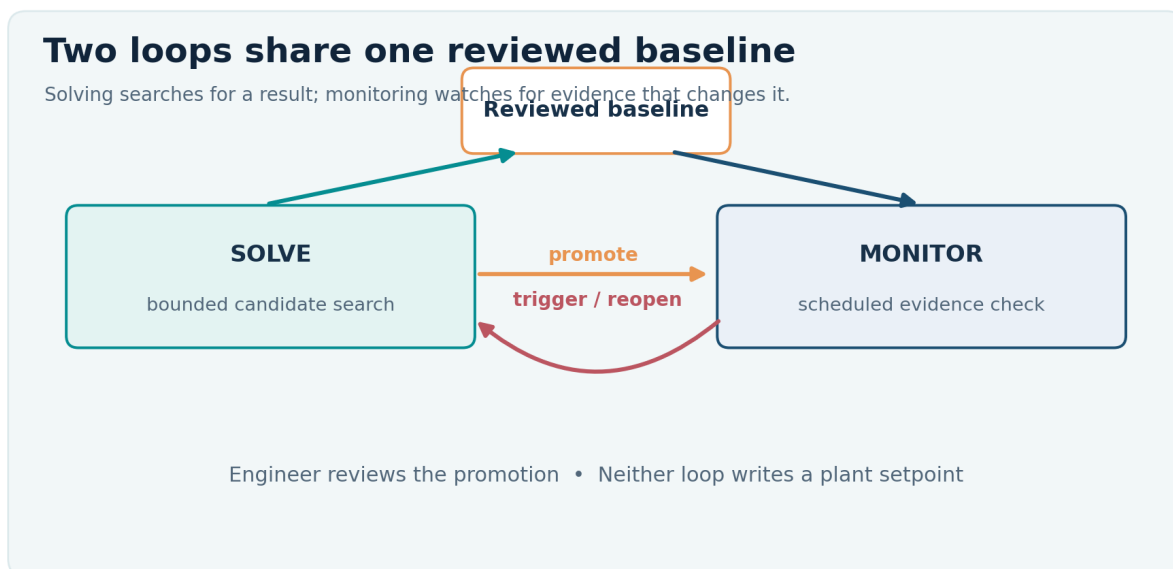


Figure 1.3: A living task has a fast bounded solve loop and a slower monitoring loop. Promotion crosses a human review boundary.

1.2 The goal contract

“Improve compression” is a useful motivation but an incomplete goal. A checkable objective states a metric, direction, unit, target, confidence requirement and comparison basis. Constraints protect the feasible operating region. A bounded search needs a budget and honest stop states. For example: “Reduce modeled compression power by at least 2.0% relative to the accepted baseline while maintaining discharge pressure and a suction temperature above 22 °C; verify the result under a specified gas composition and uncertainty range.” The number and temperature here are teaching assumptions.

The goal is initially drafted from a Word or Markdown brief. It is confirmed by a named reviewer before task-solve operates on it. The metric must be produced by a stage script; a target written in a document does not become a measurement by being parsed. The brief hash helps detect a later change in intent.

For a process problem, write the objective as $J(x; d, m)$, where x contains the decisions, d the measured or assumed boundary conditions, and m the model and its version. A feasible candidate belongs to a set $\mathcal{F}(d, m)$ defined by equipment limits, process constraints and the model’s applicable range. These symbols make an often hidden distinction explicit: a changed J can be caused by a new action x , new evidence d , or changed software and parameters m . A campaign that cannot identify which of these changed cannot attribute the gain.

The objective also needs a reference. “Power reduction” might mean relative to the first model run, yesterday’s measured power, or a controlled baseline at matched throughput. Each answers a different question. For the synthetic station in this book, a model comparison at matched pressure ratio is $100(P_0 - P_x)/P_0$ percent, with both powers calculated from the same composition and flow basis. A measured post-implementation saving would require a separate normalization of operating conditions and an observation period. Keeping the two statements distinct is essential to scientific reporting.

Chapter 2

The architecture and its boundaries

A living task adds continuous/ to the existing study folder. The Python runner records cycles, handles sources and stop rules, and creates the living report. Task specific stage functions connect evidence to NeqSim models. Java ProcessAutomation gives a stable interface to process variables and trial evaluation. Java classes under `neqsim.process.operations.continuous` provide a model-side improvement cycle, drift monitor and baseline comparator. An agent can investigate triggers through the public continuous-improvement role and its skill. Other adapters can be installed as separate Python packages.

Four improvements may arise from the same task: an operational opportunity, a better model or calibration, a repaired data source, and a change to the software or agent instructions. Record them separately. Operational value is judged against actual outcomes and constraints; model quality against withheld observations; data quality against source provenance; software quality against regression tests and compatibility. Conflating these measures allows a polished report to hide a worsening physical result.

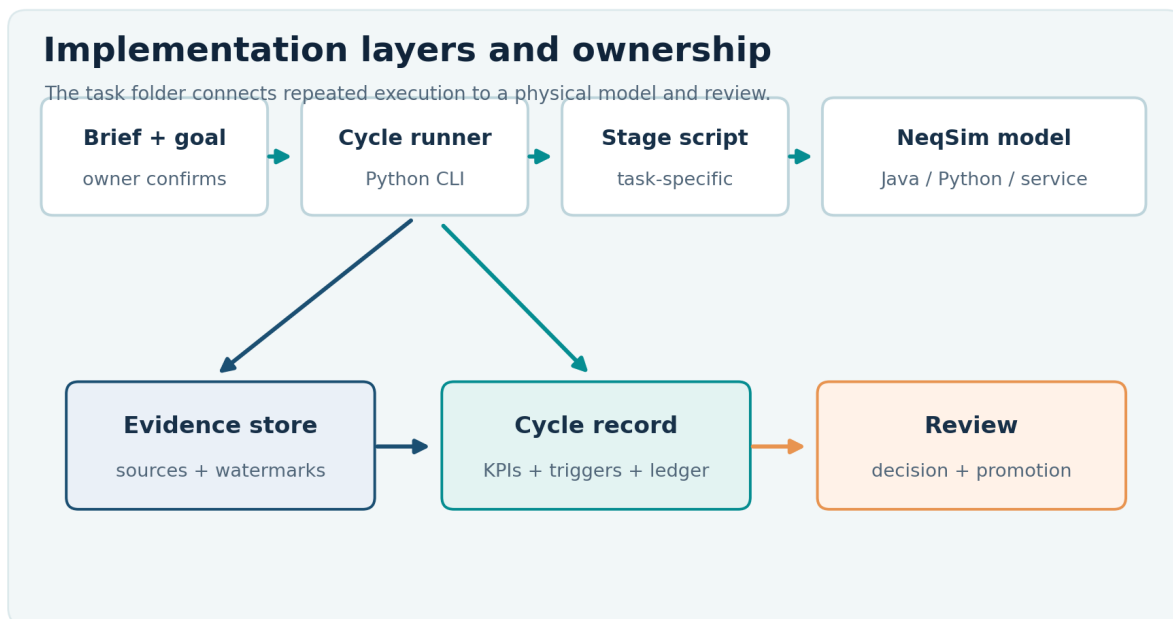


Figure 2.1: The runner controls time and records; the model computes; the agent handles exceptions; the reviewer controls acceptance.

2.1 What is implemented and what is a pattern

Layer	Current public implementation	Boundary
Task CLI	task-living, task-cycle, task-solve, task-backtest, task-schedule, task-promote, task-ledger, task-status, task-report, task-reference-case	Use the current checkout; availability in released packages may differ.
Python runner	devtools/neqsim_continuous/, including plans, sources, watermarks, ledger, state, reporting and agent launch	Task stage code and data adapters are supplied per study.
Java model side	ImprovementCycle, ModelDriftMonitor, BaselineComparator and ProcessAutomation	Model adequacy depends on the selected fluid and process representation.
Public agent and skill	.github/agents/continuous-improvement.agent.md and .github/skills/neqsim-continuous-task-improvement/SKILL.md	Agent launch is optional and configured separately.
File source	Built in CSV file adapter	Historian and business system connectors are optional external packages.
Scheduled software campaign	Issue, branch, test, draft PR and CI review workflow	An orchestration pattern, not a single NeqSim task-* command.

The Model Context Protocol (MCP) is one way to expose calculations to a tool using agent. It is not the task store or scheduler. A stage may call a NeqSim process object directly, use a tested Python/Java API, or call a governed service where available. The same cycle record should identify which capability actually ran. A missing connector records degraded evidence rather than inventing measurements.

2.2 Core, community and private extensions

The public [NeqSim repository](#) owns the numerical engine, task CLI, living task runner, core agent, core skill and public reference case. The public [community skills](#) repository publishes reusable methods and installable packages; the public [community agents](#) repository composes skills into workflows. A company may keep its own private skill and agent catalogs for approved data connectors, policies and site specific knowledge. The public book needs no access to those private catalogs and contains no private interface names or data.

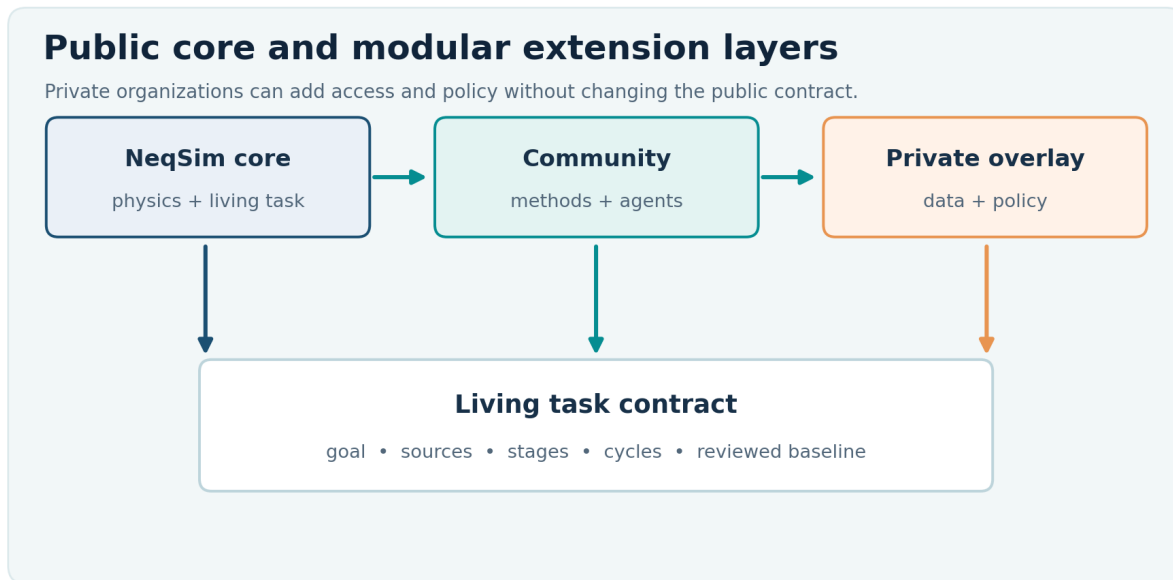


Figure 2.2: The dependency direction is outward. The public core remains useful with a CSV file source; private overlays supply access and policy without moving private content into public examples.

Repository layer	It owns	A living task uses it for
NeqSim core	Physics, CLI, cycle runner, ProcessAutomation, model side continuous classes, task format	A reproducible calculation and persistent cycle records.
Community skills	Public methods, optional Python packages, adapters and validation guidance	Plant agnostic methods such as the continuous improvement toolkit.
Community agents	Public workflow specifications and skill composition	Exception triage and a reusable continuous task improvement role.
Private organization extensions	Governed connectors, internal policy and task specific approvals	Evidence access and review rules available only to authorized users.

The public continuous-task-improvement-agent names the core continuous skill and the community toolkit among its required skills. It prepares goals, plans, backtests, triggered diagnostics and promotion recommendations; it does not accept ledger items or promote a baseline. The community `neqsim-continuous-improvement-toolkit` adds an optional historian adapter, Gaussian process expected improvement for bounded trial selection, an ensemble Kalman parameter update, and an SVD identifiability check. These are callable helpers, not a guarantee that any particular asset model or calibration is valid. The toolkit's predicted improvement is a forecast; a NeqSim evaluation and separate checks establish a realized result.

An organization can register its private catalogs through the public [enterprise repository integration guide](#). The recommended dependency direction is NeqSim core → public methods → private skills → private agents. Such a catalog is an extension point, not a prerequisite for the synthetic case. Keep the interface contract, units, missing-source behavior and approval rules visible to a reviewer even when the underlying records are private.

2.3 One task folder, many cycles

my_study/	
study_config.yaml	existing task configuration
results.json	first study result
step1_scope_and_research/	basis and sources
step2_analysis/	runnable model and checks
step3_report/	reviewed initial report
continuous/	
goal.yaml	confirmed objective and stop rules
cycle_plan.yaml	sources, stages and triggers
stages/*.py	task specific model code
baseline/	promoted KPIs and history
cycles/<cycle-id>/	one inspectable cycle
backtests/<run>/	isolated historical replay
ledger/events.jsonl	append-only decisions
data/<source>/	pulled evidence
state.json	watermarks.json
LIVING_REPORT.md	drift_state.json
report/kpi_trends.png	regenerated summary

The existing study remains the starting basis. The continuous/ layer does not rewrite its source evidence. Each cycle has a UTC time and host in its identifier. A second run in the same minute receives a suffix. State includes a lock so only one cycle runs at a time for a task. Completed stages are skipped when a crashed cycle is resumed. Keep a task folder's large data outside public source repositories.

Chapter 3

Goals, stop states and the improvement ledger

The brief should be understandable without the software. State the decision, the success metric, the applicable range of inputs, hard constraints, allowed levers, review owner, budget and evidence needed to claim success. Then map that brief into `goal.yaml`. A goal with an unconfirmed target remains a draft.

```
confirmed_by: "A. Engineer"
objective:
  metric: power_reduction_pct
  direction: maximize
  target: 2.0
  confidence_required: medium
constraints: ["suction temperature >= 22 C"]
stop:
  converged: {window: 3, relative: 0.02, absolute: 0.01}
  budget: {iterations: 12, agent_sessions: 6}
  expected_improvement_cost: 0.01
reopen:
  regress_margin: 0.5
  on: [new_evidence, brief_changed, neqsim_capability, reviewer_request]
```

This is a public teaching configuration modeled on NeqSim's current guide. `regress_margin` uses the objective's own units; a value of 0.5 here means half a percentage point, not a relative 50%. Constraints written as prose still need explicit enforcement inside a solver stage. Do not set `validated: true` merely because a simulation returned a number: the stage must establish convergence, units, constraints and the required comparison.

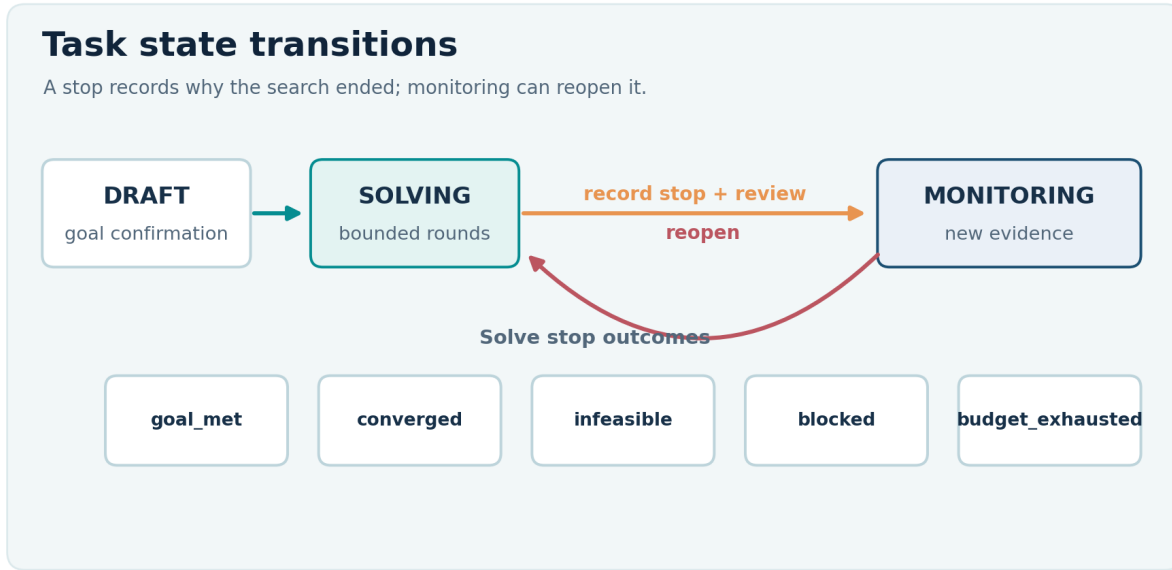


Figure 3.1: Every solve stop enters monitoring. A trigger can reopen it; promotion still requires review.

3.1 Termination states

goal_met means the target was reached with the required confidence and validation. **converged** requires recent gains below the tolerance **and** no remaining candidate with worthwhile expected net improvement. **infeasible** requires a defensible reachable bound below the target. **blocked** means the remaining action needs a person or unavailable evidence. **budget_exhausted** means the configured round or agent session budget ran out. None should be silently reported as success.

For a maximize objective with current value J , the convergence window compares new gains with $\max(\epsilon_{\text{abs}}, \epsilon_{\text{rel}}|J|)$. An expected next action can be screened by $p\Delta J - c$, where p is its chance of success, ΔJ the predicted gain and c the effort expressed in the same value units. This is a decision heuristic, not an economic result unless its inputs are calibrated.

The ledger stores proposed improvements as events. A proposed action can move through review, acceptance, implementation and verification. **model**, **data** and **tooling** categories keep different outcomes visible beside operational and maintenance opportunities. If a ledger item names a KPI, later cycles record that KPI so a reviewer can check whether an implemented action produced the predicted effect. A verified result becomes evidence; a proposal alone does not.

3.2 Worked termination calculation

Suppose a teaching solve has a current objective $J = 2.10$ percentage points of modeled power reduction, a target of 2.00 points, an absolute convergence tolerance of 0.01 point and a relative tolerance of 0.02. The gain threshold is $\max(0.01, 0.02 \times 2.10) = 0.042$ point. Three successive gains of 0.03, 0.02 and 0.01 point lie below that threshold. This alone does not justify **converged**: a candidate with expected net gain above the configured cost remains worth testing. Conversely, a result above the target does not justify **goal_met** if its constraint check failed or its confidence fell below the required level. The two tests answer different questions: has the target been demonstrated, and is further automatic search worthwhile?

An **infeasible** conclusion needs a defensible physical bound below the target. Twelve unsuccessful search rounds do not establish one. If the bound relies on unverified heat-transfer limits or motor curves, state the assumptions and use **blocked** or **budget_exhausted** until the evidence improves.

Chapter 4

Run the public reference case

The public reference case is a synthetic compressor station. Its archived CSV data include injected efficiency decline, flow meter bias and suction pressure change. The case is intentionally small: it teaches the interfaces and backtest mechanics without claiming a real facility result. The current guide reports three expected events detected and no false alarms for its recorded run; treat that as a reproducibility target for the cited snapshot, not a guarantee for changed code or thresholds.

Use a current source checkout and a Python environment with the developer tools. From the repository root, inspect `neqsim --help`, `neqsim task-status --help` and the current continuous task guide. If the console launcher is unavailable, invoke `devtools/neqsim_cli.py` with the selected Python interpreter. A model backed stage also needs the Java runtime and a compiled NeqSim checkout. The public CSV case itself does not require a company data adapter.

```
neqsim --show-task-root
neqsim task-reference-case
neqsim task-status reference_compressor_station
neqsim task-backtest reference_compressor_station \
  --start 2025-10-02 --end 2026-09-30 --repeat
neqsim task-cycle reference_compressor_station \
  --now 2026-09-30T05:00:00Z --no-agent
neqsim task-solve reference_compressor_station \
  --until converged --no-agent
neqsim task-ledger reference_compressor_station list
```

Open `continuous/LIVING_REPORT.md` inside the created task. Inspect the current goal, promoted baseline, latest KPI values, triggers, solve rounds, ledger and next actions. Then inspect `continuous/backtests/<run>/backtest_report.md` for detected and missed expected events, delay, false alarms and repeatability. Backtesting is isolated from the live watermarks and drift state. If an example result differs, record the checkout revision, selected Python, source data, plan and diff rather than adjusting a claimed result to match the book.

The [public guide](#) documents the following outcome for its reference configuration. This table reports the guide's run, not a new execution made for this edition.

Injected event	Reported detection delay	Interpretation
Efficiency decline	13 days	A gradual change accumulates evidence more slowly.
Flow meter bias	2 days	A step change crosses the configured rule after confirmation.
Suction pressure step	2 days	The step or criterion rule responds to a crossing.
Quiet periods	0 false alarms in the recorded replay	A property of the selected synthetic period and thresholds.

The reference solve is reported to stop at converged after eight rounds when run with `--until converged`. The precise result is sensitive to the plan, synthetic dataset and code revision. Repeating

the run gives a software regression check, not a performance guarantee for an operating station. A useful extension is to hold out one injected scenario while tuning thresholds on the others, then inspect whether the detector still gives a tolerable alert burden.

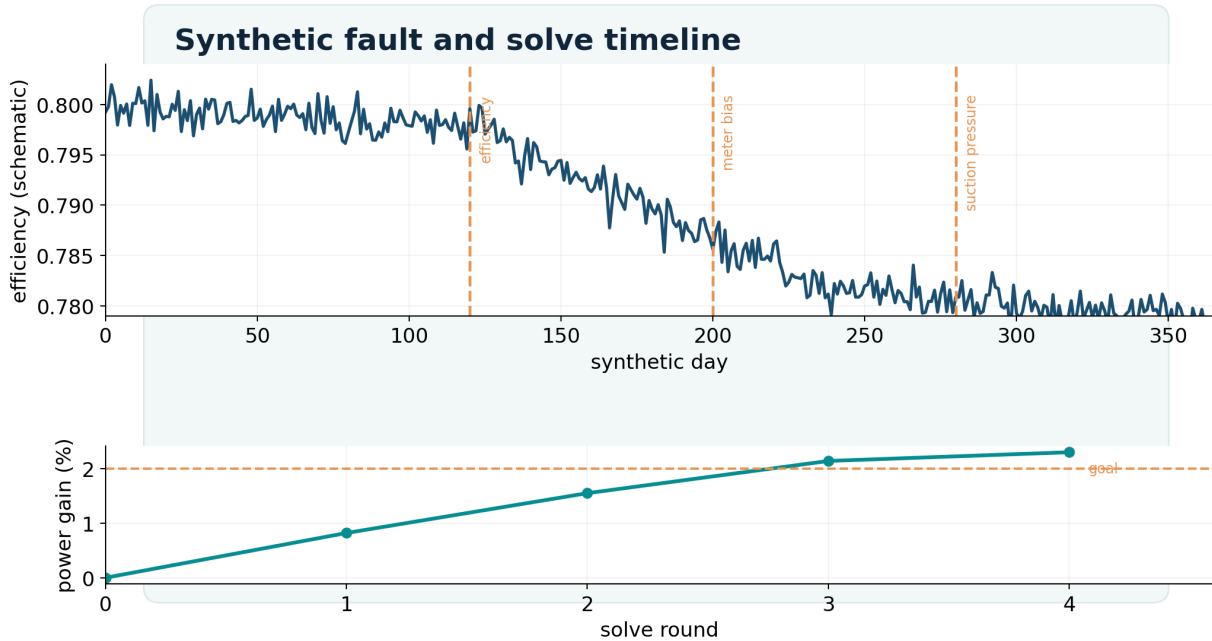


Figure 4.1: Illustrative reference case timeline. The plotted values are schematic; run the commands above for the actual saved backtest metrics.

The reference case contains a solver stage that lowers suction temperature toward a constraint in successive rounds. It reports objective values and a reachable bound. This simple strategy exists to show a terminating loop. It is not a compressor design recommendation. In a real case, the solver stage must call the accepted process model, assess bounds and uncertainty, and reserve operational changes for the plant’s decision process.

Chapter 5

Make an existing study living

Start with an ordinary NeqSim task whose `results.json` contains an inspectable first answer and whose model or analysis can be run again. A Standard task basis is useful before unattended monitoring: scope, sources, runnable work, assumptions, data gaps, initial report and work record. The scheduled command's `--standard-first` gate checks this basis and records missing elements; it does not validate the physics of the model.



Figure 5.1: An engineering record joins source measurements, model inputs and review. The visual is conceptual; the stage contract below defines the actual interface.

```
neqsim new-task "Compressor efficiency study" --type B \  
  --prompt-file brief.md  
neqsim task-living <task> --brief brief.md  
neqsim task-status <task>
```

`task-living` scaffolds a continuous/ directory without overwriting existing files. It drafts a goal from the brief, constructs a baseline from numeric `key_results` in the first `results.json`, seeds ledger proposals from its recommendations, and marks the task as living in `study_config.yaml`. Review the generated values. A populated field is not necessarily the correct engineering definition.

5.1 Define sources and KPIs

The plan is declarative; task specific physics belongs in a script. A file drop is the built in source and accepts CSV data with a time column. Each source maintains a watermark so a failed pull can

retry its window. Relative paths resolve against the task directory. Use a separate `data_root` when data volume makes the task folder unsuitable.

```
sources:
  station:
    adapter: file
    options: {folder: data_drop, time_column: timestamp, pattern: "*.csv"}
    initial_lookback_days: 1
    stale_after_hours: 48
kpis:
  suction_pressure_bar:
    {source: station, column: suction_pressure_bar, agg: mean}
scripts:
  station_model:
    {file: continuous/stages/station_model.py, function: run}
stages: [sense, refresh, "script:station_model", kpis, drift,
  goal, diff, ledger, digest, notify, agent]
```

The available aggregations include mean, min, max, last, sum and count. A script can return additional KPIs. Name each KPI with its unit where possible and keep the underlying raw unit in source metadata. Watermark advancement means “these rows were collected”, not “the model is valid”. A missing plugin records `not_installed` and degrades the cycle.

5.2 Write a stage with a checkable contract

The stage receives `ctx.new_rows`, the goal and plan, promoted baseline, task and cycle directories, current clock, mode and dry run flag. It returns a dictionary of KPIs, proposals, triggers, status and optionally solve information. The following deliberately small stage checks its inputs and calculates an average pressure. A NeqSim process study replaces the teaching calculation with a model evaluation, while preserving the return contract.

```
def run(ctx):
    rows = ctx.new_rows.get("station", [])
    pressures = [row.get("suction_pressure_bar") for row in rows]
    valid = [value for value in pressures if isinstance(value, (int, float))]
    if not valid:
        return {"status": "warn", "message": "no valid pressure rows"}

    mean_pressure = sum(valid) / len(valid)
    if not 0.0 < mean_pressure < 500.0:
        return {"status": "fail", "message": "pressure outside study range"}
    return {"kpis": {"suction_pressure_bar": mean_pressure}}
```

In a NeqSim backed stage, load a model with an explicit composition, selected EOS and input basis. Apply measured values only to variables designated as inputs. Run `ProcessAutomation.evaluate(...)` for candidate setpoints, inspect feasible and rejected writes, then read KPIs and a utilization snapshot. The `ImprovementCycle` Java class wraps one such evaluation, drift update and comparison and returns schema versioned JSON with feasibility, KPI, drift, comparison, triggers, bottleneck and errors. It does not choose the correct objective or approve a plant action.

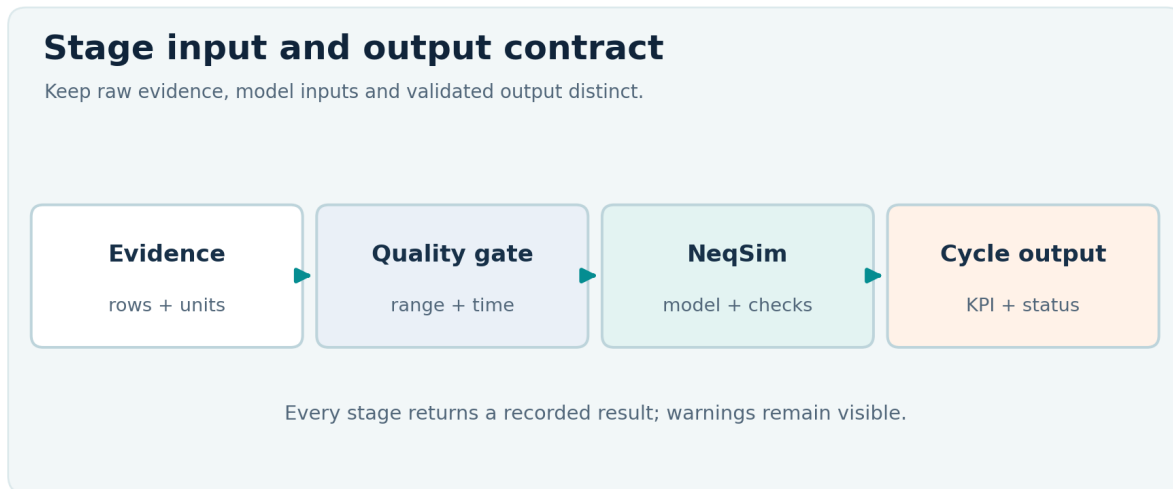


Figure 5.2: A stage is the explicit seam between facility data and a physical calculation.

5.3 Bound and validate a solve stage

A solver stage reports its current objective value, whether that value passed its checks, a confidence label, an attainable bound if one exists, and candidate next actions. One possible return shape from the current guide is:

```

return {
  "kpis": {"power_reduction_pct": value},
  "solve": {
    "value": value,
    "validated": checks_passed,
    "confidence": "medium",
    "action": "evaluate a cooler suction case",
    "upper_bound": reachable_bound,
    "candidates": [{
      "action": "test next suction temperature",
      "branch": "cooling",
      "p_success": 0.8,
      "predicted_delta": 0.2,
    }],
  },
}

```

The values are computed in the script; this fragment only shows the interface. `checks_passed` should include convergence, constraints, nonzero physical outputs, mass or energy checks when relevant, and an independent or withheld comparison proportional to the claim. If the model cannot establish a reachable bound, omit it rather than inventing infeasible. Use the plan's `solve.stages` and `round budget` to restrict the repeated work.

Chapter 6

Qualify data, detect change and back-test

A persistent cycle does not make poor inputs reliable. Every source row needs a timestamp, unit, provenance and status. Check range, missingness, duplicate timestamps and time alignment before feeding it to a model. If measurements are reconciled, keep the unadjusted values and the residuals. Assign inputs and withheld benchmarks before running the process model. A pressure copied from the historian and then “predicted” from that same value is not validation.



Figure 6.1: Conceptual monitoring scene. The traces suggest repeated observation and a change to investigate, not measured plant performance.

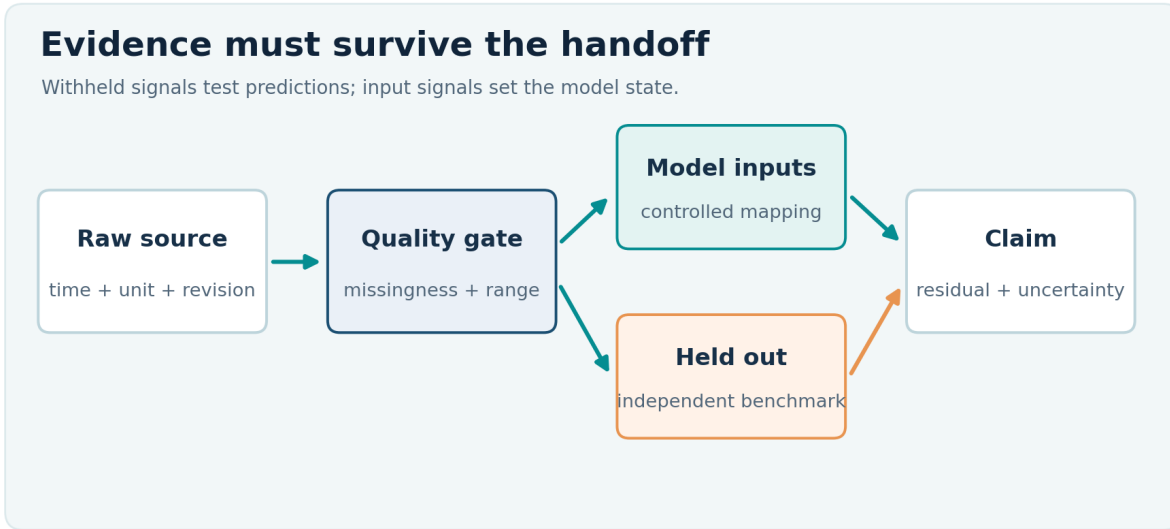


Figure 6.2: Keep source checks, model inputs and independent benchmarks separate.

NeqSim’s plan can watch simple threshold crossings and statistical drift. Its drift monitor uses an exponentially weighted moving average (EWMA) and two sided cumulative sum (CUSUM) against a frozen warm up baseline. A per signal `min_sigma` is an engineering noise floor; `confirm` requires repeated out of limit values. Without a floor, a short warm up can underestimate noise and produce false alarms. A crossing trigger fires on the transition into the state, not on every day the state persists.

6.1 What the detector computes

The current Java `ModelDriftMonitor` fixes a mean μ_0 and sample standard deviation s_0 from the first warmup finite observations. It uses $\sigma = \max(s_0, \sigma_{\min})$ when an engineering floor is supplied. For later observation x_t , the EWMA starts at $z_0 = \mu_0$ and follows

$$z_t = \lambda x_t + (1 - \lambda)z_{t-1}, \quad |z_t - \mu_0| > L\sigma\sqrt{\frac{\lambda}{2 - \lambda}}.$$

The expression on the right is the steady-state control limit implemented by the class, not a confidence interval on a physical parameter. EWMA and CUSUM are established sequential monitoring methods [9]. The standardized residual $r_t = (x_t - \mu_0)/\sigma$ also updates two one-sided accumulators:

$$C_t^+ = \max(0, C_{t-1}^+ + r_t - k), \quad C_t^- = \max(0, C_{t-1}^- - r_t - k).$$

A provisional out-of-control value occurs if either EWMA exceeds its limit or $\max(C_t^+, C_t^-) > h$. The `confirm` count requires consecutive provisional values. Once an alarm is raised, the class latches it until reset; the `newAlarm` field distinguishes the transition from a continuing condition. NaN and infinite inputs are ignored rather than treated as meaningful observations. These implementation details matter when comparing a Java client with the Python task runner, which is intended to give the same alarms for identical data and settings.

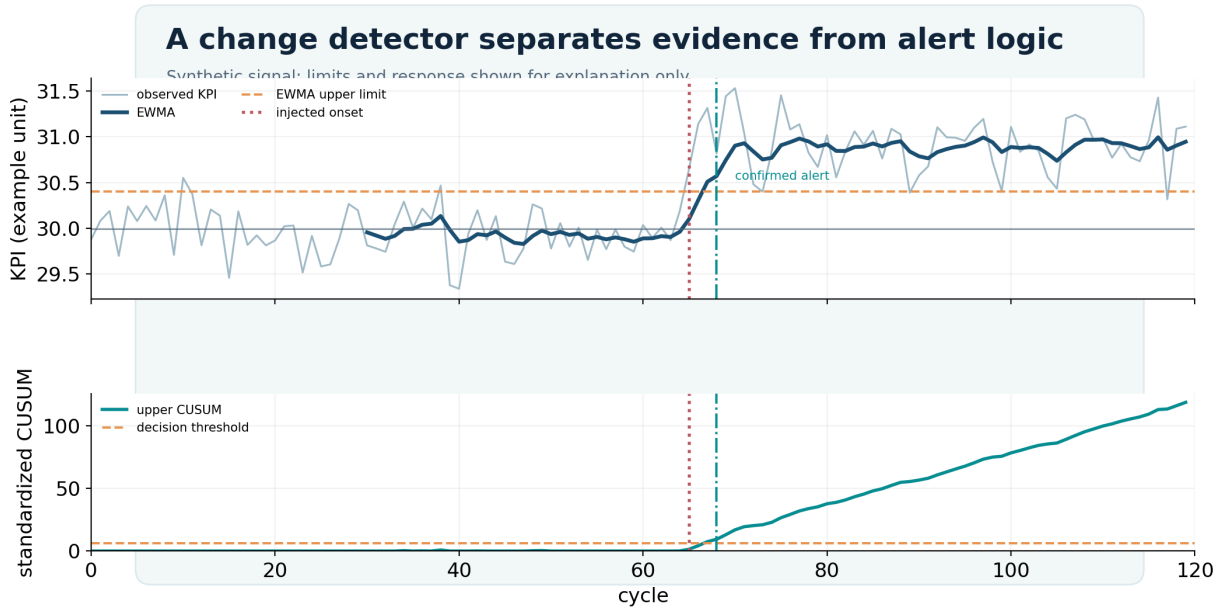


Figure 6.3: A schematic monitored KPI, its frozen reference, an injected step and confirmation before alert.

Statistical evidence and engineering relevance are separate. A very quiet synthetic series can make a small shift statistically detectable even if its physical effect is negligible. The `min_sigma` floor prevents the monitoring scale from collapsing beneath the chosen engineering resolution. It must be recorded in the KPI's unit. A 0.2 bar floor is not transferable to efficiency, and a dimensionless efficiency floor of 0.002 is not transferable to a percentage-point KPI without conversion.

drift:

```
signals: [polytropic_efficiency, suction_pressure_bar]
settings:
  lambda: 0.2
  L: 3.5
  k: 0.5
  h: 6.0
  warmup: 30
  confirm: 2
signals:
  polytropic_efficiency: {min_sigma: 0.002}
  suction_pressure_bar: {min_sigma: 0.2}
```

triggers:

```
kpi_step: {suction_pressure_bar: 2.0}
criteria: {polytropic_efficiency: "< 0.785"}
```

These thresholds are from the teaching case and are not universal alarm settings. Select floors from instrument resolution and observed variability under a stable regime. Choose the smallest sustained shift worth a review. Backtest on archived periods with known event onsets, quiet periods and data failures. Record detection delay, missed events, false alarms per month and repeatability. Change thresholds on a development interval, then evaluate on a separate interval to avoid tuning to the answer.

```
neqsim task-backtest <task> --start 2025-10-02 \
  --end 2026-09-30 --step-hours 24 --name trial_01 --repeat
```

The result is written under `continuous/backtests/trial_01/`. It leaves live state untouched. A backtest with three injected events only establishes behavior on those events; it does not establish detection rates for all future faults. For operational deployment, extend the archive, vary regimes,

include missing data and check whether an alert leads to the correct diagnostic action.

6.2 Score the backtest against a decision

Specify an event interval and the maximum acceptable delay before running the replay. A detection that occurs after the useful decision window may be a numerical success and an operational failure. Count false alerts over quiet exposure time, not merely over the number of known faults. Record a separate outcome for a stale or failed source; suppressing an alert by losing the data is not a good detector. When measured faults are unavailable, use synthetic perturbations to test plumbing and state behavior, then describe physical detection performance as unestablished.

The backtest also tests the task's temporal logic. Watermarks must not advance after failed pulls. A repeated run with the same inputs and version should reproduce event times. A model revision needs an explicit comparison with the archived revision. A threshold selected after seeing all fault dates should be evaluated on a new period, because otherwise the reported delay and false alarm rate are optimistic. The public three-fault case is best used as a regression fixture for those mechanics, followed by broader site-specific qualification where such use is intended.

Chapter 7

Run, schedule, triage and promote

Run a manual dry cycle before scheduling. Inspect the source status, the stage return, every KPI unit and the digest. `--dry-run` writes a cycle folder but does not advance watermarks, the ledger or drift state. `--no-agent` keeps the cycle deterministic when testing the data path.

```
neqsim task-cycle <task> --dry-run --no-agent
neqsim task-cycle <task> --standard-first --no-agent
neqsim task-report <task>
neqsim task-status <task>
```

The Standard first status appears in `continuous/standard_first_status.json`. A missing report or work record may be generated from the accepted task; missing model evidence is recorded as incomplete or blocked. The gate is about task readiness, not a certificate of process correctness.

7.1 Schedule monitoring

On Windows, `task-schedule` can preview, install, show and remove a Task Scheduler entry. It runs `task-cycle` in monitor mode with `--standard-first`. On Linux the command prints a cron line to install with the normal operating system scheduler; it does not install cron itself. Use an explicitly selected interpreter, pinned checkout or release, predictable working directory and a service identity with only the access needed to read sources and write the task folder.

```
neqsim task-schedule <task> --daily 05:00
neqsim task-schedule <task> --daily 05:00 --install # Windows
neqsim task-schedule <task> --show
```

For a Linux host, use `neqsim task-schedule /data/tasks/<task> --daily 05:00` and review the printed cron line before installing it. Set `NEQSIM_CONTINUOUS_HOST` to distinguish cycle IDs. Do not embed passwords in `cycle_plan.yaml`; notifier fields refer to environment variable names. Keep only one writer active per task. Separate copies on two hosts need deliberate state synchronization; the append only ledger can be merged by event ID, but two independent data and watermark histories are not magically one timeline.

7.2 Triage an exception

The cycle digest answers what changed and whether a decision is needed. Inspect `triggers.json`, `validation.json`, `diff_vs_baseline.json`, and `refresh_report.json` before treating a drift flag as a physical diagnosis. A stale source can produce a different response from a valid low flow. A new NeqSim commit can move a calculated KPI with unchanged plant inputs. Reevaluate the prior baseline inputs under the new code and compare both versions before attributing the difference to the process.

When `agent.enabled` is true, a trigger may launch a bounded headless continuous-improvement review. The public configuration uses an explicit executable, timeout and deny list. The agent can write `agent_review.md` and propose ledger items; the documented headless path denies commit, push and PR creation and does not promote the baseline. The repeated cycle itself uses no language model calls. A missing agent executable records `not_installed` while the deterministic cycle continues.

```
agent:
  enabled: true
  agent: continuous-improvement
  executable: copilot
  timeout_s: 1800
notify:
  channels: [file]
report: {formal: on_promote}
```

7.3 Review a result

The living report is regenerated after cycles, solve rounds, backtests, promotions, reopening and ledger decisions. It is a view of the recorded task, not a manually edited source of truth. A reviewer can accept or reject each ledger item separately, then promote a selected cycle. The previous baseline is archived. Promotion merges the cycle's KPIs over the prior baseline, so a solve round that reports only its objective does not erase unrelated monitored KPIs.

```
neqsim task-ledger <task> list
neqsim task-ledger <task> set OPP-0001 accepted \
  --by "A. Engineer" --note "reviewed against design basis"
neqsim task-promote <task> <cycle-id> \
  --reviewer "A. Engineer" --note "validated model case"
neqsim task-report <task> --formal
```

Promotion means “use this reviewed cycle as the next comparison basis.” It does not dispatch a setpoint or declare a process change implemented. The formal Word/HTML report can be regenerated on_promote, every_cycle or never; choose a cadence suited to the decision process.

Chapter 8

The NeqSim model side in detail

The Python runner is useful for every task type. A process model stage can additionally use the Java automation API. `ProcessAutomation` reads and writes named variables, evaluates candidate setpoints and returns diagnostics. Its `evaluate(...)` call includes feasibility and rejected setpoints; its utilization snapshot reports equipment constraints. A task may hold a `ProcessSystem` or `ProcessModel` and expose only the addresses appropriate for the study. Set temperature, pressure and flow units explicitly and use a cloned model for independent candidate branches where state carryover would bias results.

`ImprovementCycle` is a model-side wrapper around one evaluation. Construct it with a non-null automation facade, register KPI addresses with units, optionally replace its drift monitor or comparator, and call `runCycle(measuredInputs, inputUnit)`. The returned JSON includes a schema version and cycle count. For a feasible evaluation, it reads each KPI, updates drift, compares with the baseline, requests a utilization snapshot and collects errors instead of throwing from the cycle. A failed evaluation yields `model:infeasible` and should not be interpreted as a real process alarm until the input and solver failure are classified.

```
ProcessAutomation automation = process.getAutomation();
ImprovementCycle cycle = new ImprovementCycle(automation);
cycle.addKpi("Compressor.power", "kW");
cycle.getComparator().addCriterion("Compressor.power", "> 5000");
String resultJson = cycle.runCycle(measuredInputs, "bara");
```

This excerpt illustrates the public class contract, not a complete runnable plant model. Its single `inputUnit` applies to all measured inputs passed in this call, so mixed pressure and temperature inputs require an appropriate mapping or separate application path. Verify address names, setter validation, model initialization and unit conversion in the exact checkout. A uniform value map should never silently imply uniform physical dimensions.

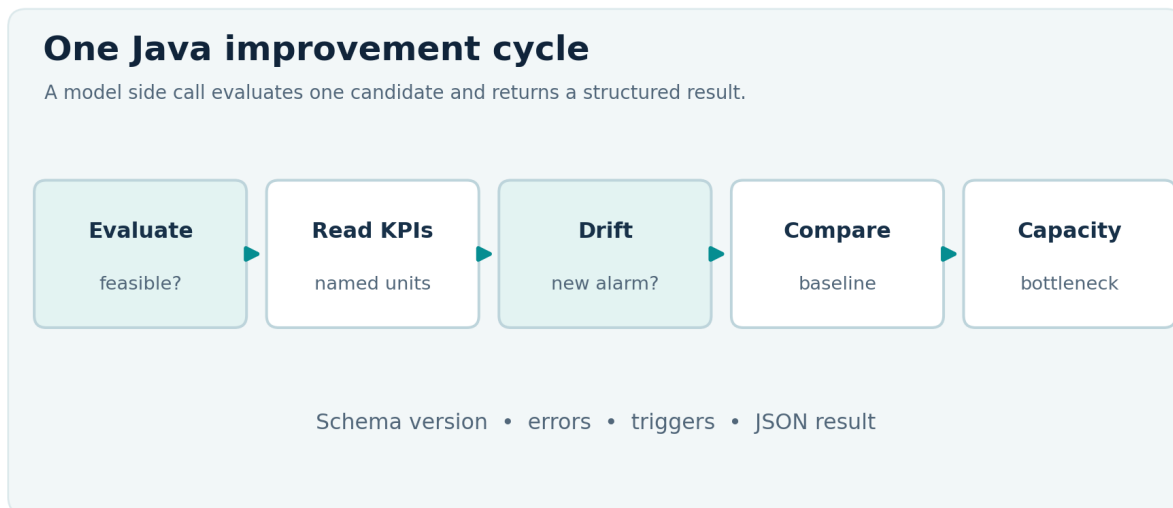


Figure 8.1: Model-side order of operations. The runner retains the records; the Java class evaluates one model cycle.

The cycle and a broader task solve answer different questions. `ImprovementCycle.runCycle` does not implement a whole campaign, prepare a report, install a scheduler or decide which candidate should be promoted. The Python task-solve loop calls task defined stages repeatedly and applies the goal and stop rules. A model stage can call `ImprovementCycle`, `ProcessAutomation.evaluate`, or other NeqSim APIs depending on the question. State which route was used in the cycle provenance.

8.1 Calibration, uncertainty and domain guards

Drift can arise from a sensor, a physical change or a model change. First test time alignment, units, tag mapping and sensor quality. Then compare withheld signals. Parameter estimation can be useful when the parameter is identifiable in the available window and the inferred shift has a physical interpretation. Do not recalibrate every time a residual appears. If a hybrid correction model is used, define its training range, features and uncertainty; outside that domain return to the physics model or mark the result unsupported. Candidate ranking should propagate uncertainty into the predicted gain, not only display a point estimate.

8.2 Choosing the next calculation

The public community toolkit provides `propose_next`, a Gaussian process surrogate with an expected-improvement acquisition over bounded variables. It proposes a new point x after earlier NeqSim evaluations $D = \{(x_i, J_i)\}$. In a maximization setting, an acquisition value can rank points by the expected improvement over the best observed objective [10]. It is **not** the improvement already achieved. The stage must run the proposed point through NeqSim, check its constraints and then return the realized $J(x)$. The toolkit's current method uses an RBF kernel after scaling the bounded inputs to a unit cube and searches a set of random candidates. Its own documentation describes it as a screening method with limited information when only a few trials exist.

This strategy is useful when one NeqSim trial is expensive and the number of controlled variables is small. A smoother, inexpensive and well-behaved process may suit a deterministic optimizer better. For either method, report the number of model calls, the best feasible point, rejected points, the uncertainty or sensitivity around the selected point, and the effect of a different initial state. An optimizer does not resolve an incorrectly specified objective or missing operating constraint.

```
from continuous_improvement_toolkit import propose_next

proposal = propose_next(
    previous_setpoints,
    previous_validated_objectives,
    bounds=[(22.0, 35.0), (40.0, 60.0)],
    maximize=True,
    seed=cycle_number,
)
candidate = proposal["x"]
# Evaluate candidate with the accepted NeqSim model and constraints.
```

The bounds in this example are illustrative and the variables have deliberately been left unnamed; a real stage must identify their units and allowed meaning. The `expected_improvement` output belongs in a proposal or stopping heuristic, not a measured KPI. Persist the trial dataset, random seed and surrogate settings so another run can explain a changed recommendation.

8.3 Parameter learning and identifiability

The same toolkit offers `enkf_update` for an ensemble of uncertain parameters [11]. Its inputs are parameter ensemble members, matching model predictions for each member, a measured vector and observation standard deviations. The method updates the ensemble through cross-covariance between parameters and predicted observations. Before accepting the updated parameters, compute the sensitivity matrix $H_{ij} = \partial y_i / \partial \theta_j$ at the study state and scale its columns by physically meaningful parameter ranges. If its singular values show deficient rank, the available observations cannot determine all parameter combinations. The toolkit’s identifiability report exposes the rank, condition number and weak directions.

For example, a rising compressor power residual can be explained by lower efficiency, a biased flow meter or a changed gas composition. If only electrical power is observed, the parameters may trade off and the calibration can fit power perfectly while identifying the wrong cause. A withheld discharge temperature, independent flow measurement or fresh composition can discriminate among explanations. If none is available, fix a parameter from an independent source, report a family of plausible states, or leave the cause unresolved. The community documentation recommends at least 20 ensemble members for its update and notes its approximate Gaussian error assumption; an application still needs its own sensitivity and posterior predictive checks.

Learning also changes the baseline’s meaning. Keep the measurements used for fitting separate from measurements used to judge prediction. Record the prior, observation uncertainty, accepted parameter range, model revision and time window. A lower training residual is an estimate of fit, not a demonstrated improvement in process performance. The living task ledger can record the new model state as a model proposal until a reviewer accepts the independent evidence.

Chapter 9

A complete teaching walkthrough

Suppose a synthetic three component gas is compressed in a small station. The first study has an accepted `results.json` and a model script. A separate CSV contains hourly suction conditions. The question is whether power can be reduced by lowering inlet temperature while respecting a dew point margin and a motor limit. The aim is a method the reader can adapt; no result below describes a real asset.

Step 1, define the decision. Write the target relative to a fixed baseline, the source and revision of the gas composition, the pressure requirement, the temperature constraint and the review owner. Specify which result would show a failed attempt: model infeasibility, insufficient improvement, unsupported uncertainty or unavailable measurements. Create the task and make it living.

Step 2, keep input and validation signals distinct. The model uses measured inlet pressure, temperature and flow as inputs. Discharge temperature and electrical power are withheld benchmarks. The stage records input quality, predicted power, residual against a measured value when available, equipment utilization, convergence diagnostics and the model revision. If the flow meter has a bias, its use as an input can explain a power residual without implying compressor fouling.

Step 3, implement the bounded search. Candidate inlet temperatures stay above the stated minimum and within the cooler's feasible region. Each trial starts from a known model state, calls `evaluate`, checks feasibility and reads power in kW. A solver stage returns the best validated power reduction, a reachable bound if defensible, and the next test. If the physical margin is unknown, the stage returns blocked rather than testing a lower temperature.

Step 4, replay and monitor. Use archived synthetic data with known anomalies to tune the detection delay and false alarm cost. Run a dry cycle, then a scheduled monitor. A drift trigger initiates a diagnostic review: check data health, rerun the same baseline through the current model version, compare held out signals, and record competing explanations. The agent can propose a sensor inspection, recalibration or a new scenario, each in its own ledger category.

Step 5, review and promote. The engineer inspects the cycle artifacts, verifies that constraints and withheld comparisons passed, decides any action through the normal facility process, and promotes only the accepted model result. Later cycles compare to that baseline. An action marked implemented is not marked verified until observed KPIs support the claim after the change.



Figure 9.1: A task can remain useful after its first report because evidence and decisions are retained across cycles.

9.1 A minimal inspection checklist

Question	Where to look
Which brief and goal were used?	study_config.yaml, continuous/goal.yaml, brief hash
What arrived in this cycle?	refresh_report.json, source data and watermarks
Did the model run correctly?	Stage result, cycle.json, convergence and feasibility fields
Did any rule fire?	triggers.json, validation.json, diff_vs_baseline.json
Is the result accepted?	Ledger decisions, reviewer and baseline history
Can someone repeat it?	Code revision, model inputs, plan, runtime and cycle artifacts

Chapter 10

Continuous improvement of NeqSim itself

A living study can reveal a missing model, a zero or nonfinite property, an API that cannot express an engineering constraint, a misleading report, a numerical instability, or a slow calculation. That observation is a software improvement candidate. It belongs in a public issue only after the behavior is reproduced with a safe synthetic input and checked against the current default branch. Site data stay in the task folder. A code change belongs in the repository that owns the capability, with focused validation and documentation.



Figure 10.1: From a reproducible engineering observation to a reviewed software revision. The displayed checks are illustrative.

NeqSim's `AGENTS.md` directs an agent to improve tooling during engineering tasks and to record changes. It requires source inspection, Java 8 compatible code, JavaDoc, formatting, tests and documentation. This is a per task quality rule. A scheduled software campaign extends it across time: revisit a declared goal, refresh the current issue and branch state, choose the next coherent change, publish reviewable evidence and continue after CI feedback. The campaign is a workflow built around GitHub and a scheduler. It is distinct from `neqsim task-schedule`, which schedules monitoring of a particular engineering task.

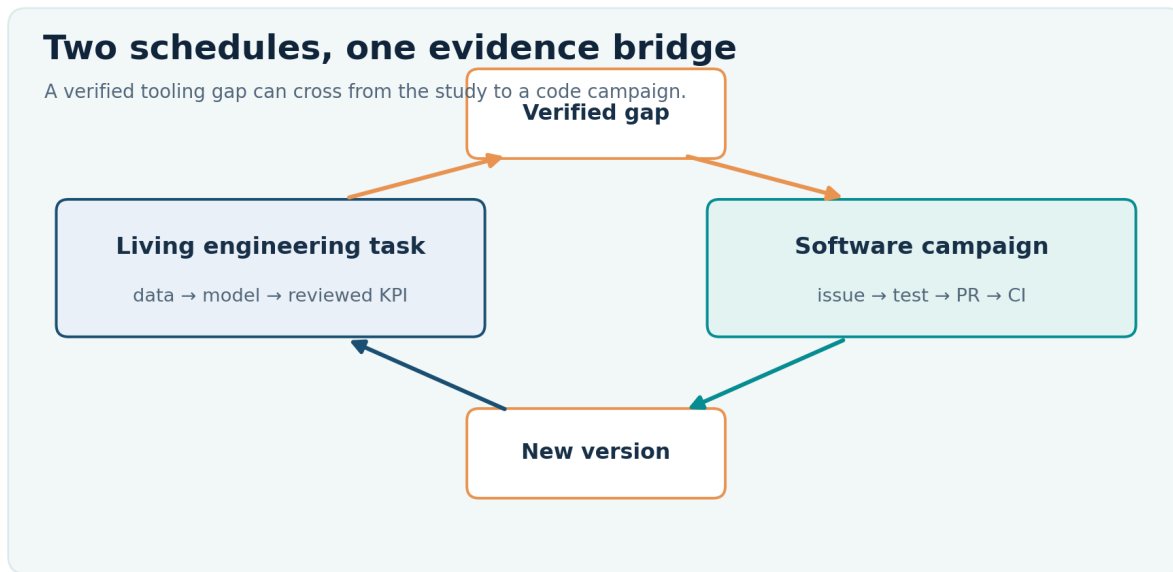


Figure 10.2: Model evidence can propose a software change; merged software can change the model prediction and trigger a new comparison.

10.1 From a goal to a bounded development lane

The campaign goal should say which engineering capability is to improve and how success will be measured. “Improve flash robustness” becomes, for example, “For these public water rich and near three phase cases, avoid phase flips under small pressure perturbations while preserving material balance, finite properties and acceptable runtime.” The campaign then carries an issue, minimal reproducer, expected physical bounds, tests, benchmark evidence, branch and PR. A schedule supplies repeated opportunities to advance the work; it is not an instruction to produce a new PR on every run.

The campaign begins each run by reading the current default branch, issue discussion, open and merged PRs, checks and review comments. It chooses a coherent change within the declared scope. It must avoid duplicate fixes already merged and must not claim that an infrastructure blocked test passed. If a local build is unavailable, a draft PR can state exactly what was validated and what is pending CI. A failing CI job or review comment becomes the next input to the campaign.

10.2 The issue to PR cycle

1. **Observe:** Capture a reproducible discrepancy from a public or reduced input. Name NeqSim version, model, units, actual and expected behavior.
2. **Classify:** Distinguish a library defect from user script error, unsupported model range, missing data or an unimplemented feature.
3. **Bound:** Set acceptance tests and physical invariants before editing. Consider nearby algorithms and compatibility.
4. **Implement:** Change the owning API or algorithm; add focused tests, a benchmark where performance is claimed, and updated docs or examples.
5. **Validate:** Run formatter and relevant tests, inspect numerical results, compare against an independent bound, then read CI on the exact PR head.
6. **Review:** Publish a draft or ready PR according to the evidence. Answer review comments and repair failures within scope.
7. **Merge and learn:** Merge through repository governance, close linked issues when the

merged behavior actually meets acceptance criteria, and update skills or instructions with reusable lessons.

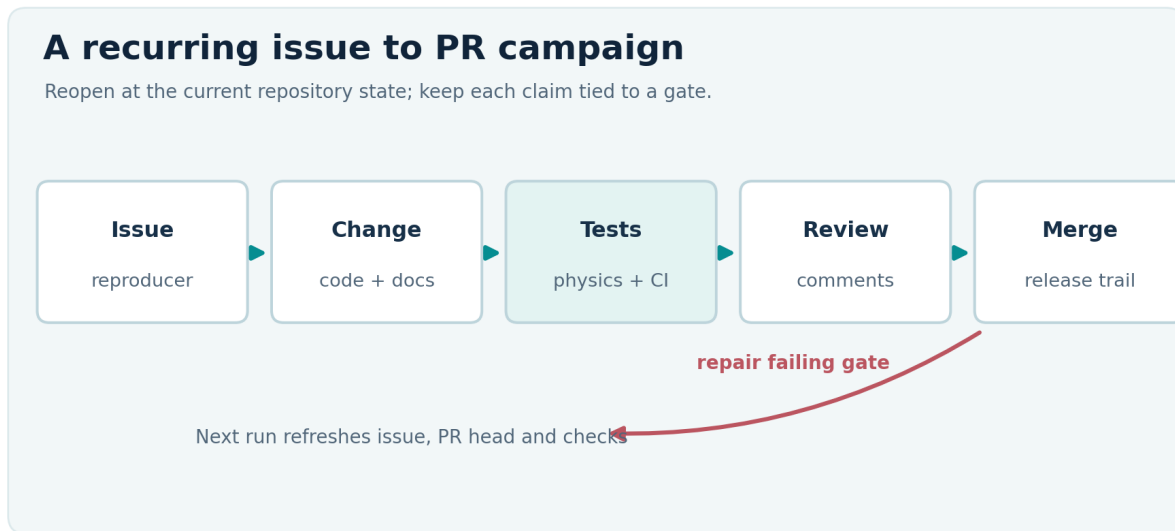


Figure 10.3: The schedule restarts the workflow at the current repository state; it does not bypass CI or review.

10.3 A campaign record

A useful run ledger stores issue ID and goal, current branch and exact head, open PR, CI status, tests actually run, remaining benchmark gaps and the next action. Each entry should distinguish **implemented**, **tested locally**, **CI passed**, **reviewed**, **merged** and **released**. Those are separate states. A branch may contain a fix that is not in master; a passed unit test does not prove physical accuracy; a merged change may not yet be in the packaged version used by a task.

campaign:

```

goal: "stabilize water rich TP flash near a phase boundary"
repository: "equinor/neqsim"
issue: "public issue URL"
acceptance:
  - "finite phase properties and material balance"
  - "no unphysical branch flip under a bounded perturbation"
  - "representative regression and runtime comparison"
cadence: "periodic, bounded engineering run"
next_run_reads:
  - "default branch and issue comments"
  - "open PR head, reviews and CI"
  - "previous run ledger"
  
```

This is an illustrative campaign specification, not a built in NeqSim configuration file. A schedule can be implemented in an approved automation host, CI platform or task service. Its authority and credential scope must be explicit. For public repository work, use the repository's contribution process and preserve review gates. A campaign should aim to finish the current coherent PR before opening another overlapping branch.

10.4 A diagnostic example for a code campaign

Consider a living property task that suddenly reports a zero liquid density. The first campaign step is **classification**, not editing the equation of state. Repeat the calculation with a minimal public fluid and explicit (T), (P), mixing rule, phase selection and density unit. Check that the phase exists and that properties were initialized after the flash. Compare the exact old and new NeqSim revisions on identical inputs. If the zero arose because the caller omitted `initProperties()`, repair the example, skill or API diagnostic. If the public API itself returns a zero or placeholder after a valid calculation, write a regression that fails for that pathway and investigate the implementation. The proposed remedy depends on this evidence.

The campaign goal might be: “Every documented liquid density pathway returns a finite, physically plausible value or a classified error for the public regression cases.” It should include at least one independent density reference or defensible bound, a case near a phase boundary, tests for unit selection and an existing case that must remain unchanged. A positive finite assertion alone is too weak: a value can be finite and physically wrong. A CI gate can enforce the structural contract, while benchmark evidence addresses model fidelity. When the fix changes a public API or default, the affected guide and example need revision on the same branch.

Evidence item	What it establishes	What it does not establish
Minimal reproducer and first failing test	The reported path is testable on an exact revision.	The root cause or generality of the failure.
Focused regression after a fix	The selected path now meets its asserted contract.	Accuracy across all fluids and regimes.
Independent property comparison	Fidelity over the stated validation points.	Fidelity outside the model’s applicability.
Broader suite and formatter	Compatibility with nearby software contracts.	Absence of all numerical regressions.
PR checks and review	The exact branch head passed the recorded gates.	Availability in a released package or approval for a facility.

At the next scheduled run, read the PR’s actual head and check results. If CI has failed, repair the demonstrated failure before extending scope. If the issue is already resolved on the default branch, run the reproducer there and record why the campaign is complete. If the accepted source change shifts a living task’s KPI, rerun its archived baseline on both revisions as described in Chapter 11. This is a closed learning loop, but it does not require an agent to merge code or alter an operational baseline automatically.

10.5 What a scheduled agent can learn

The durable learning is a test, a benchmark, a documented API pattern, a skill correction or a more informative failure classification. A private chain of thought is not a stable engineering record. An agent that fixes a phase root selection should leave an executable regression and a note explaining the physical regime. The next run reads that evidence and avoids rediscovering the same defect. This is closely related to feedback based agent research, but an engineering loop must validate against physical and software contracts, not merely verbal self assessment.

Chapter 11

How the two kinds of schedule connect

The **task schedule** asks whether a particular study remains supported by current evidence. The **software campaign schedule** asks whether a reusable NeqSim capability has met its development goal. Both benefit from a persistent state and stop criteria, but they should not share authority. A monitoring agent should not silently patch library code or push a PR because a KPI drifted; a coding agent should not update a plant setpoint because CI passed.

The bridge is a verified improvement item. A task's tooling ledger item can link a public issue with a minimal synthetic reproducer. A merged fix changes the NeqSim commit; sense detects a capability change, so the living task can replay unchanged baseline inputs on the new revision. It then asks whether the difference is a software effect or new process evidence. This closes a feedback path without confusing software correction with production benefit.

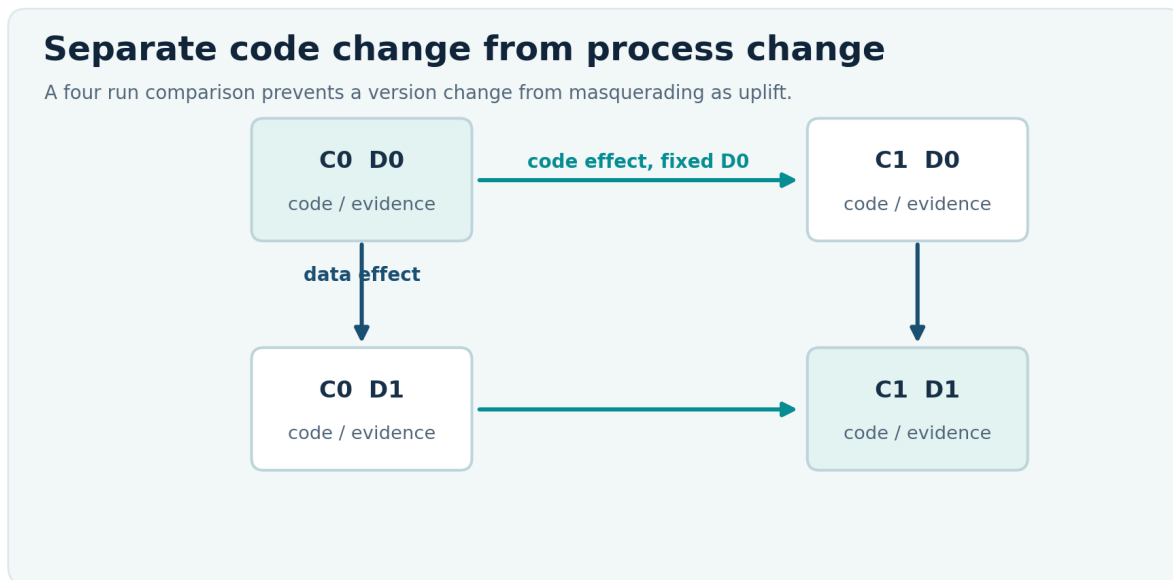


Figure 11.1: Hold inputs fixed when testing a code change; hold code fixed when testing a process change.

Change observed	First comparison	Appropriate action
New historian value, unchanged code	Source quality and model residual	Investigate data or process; continue monitoring.
New code, unchanged archived input	Old and new model result	Validate numerical change and update model provenance.
Both changed	Four combinations of old/new code and old/new input	Separate software effect from evidence effect before a decision.

Change observed	First comparison	Appropriate action
New constraint or revised brief	Goal and applicable basis	Review the objective; reopen solving with explicit scope.

11.1 A four run comparison

For a KPI y , evaluate $y(C_0, D_0)$, $y(C_1, D_0)$, $y(C_0, D_1)$ and $y(C_1, D_1)$, where C is code and D is the evidence set. The difference $y(C_1, D_0) - y(C_0, D_0)$ isolates a code effect on the old data; $y(C_0, D_1) - y(C_0, D_0)$ isolates a data effect under old code. Nonlinear interactions remain possible, so also inspect the fourth case. Record model settings and convergence status for all four. This simple experimental design can prevent a software update from being misreported as a physical improvement.

Chapter 12

State of the art and evidence of progress

Current agent research demonstrates useful patterns: interleaving reasoning and tool actions, retaining feedback across attempts, and evaluating agents on real repository issues. The engineering application adds constraints those benchmarks do not settle: dimensional units, physical validity, provenance, persistent operating data, false alarm costs, independent model checks and responsibility for a facility decision. A sophisticated optimizer can still optimize the wrong objective; a backtest can overfit a few injected faults; a successful PR can still change a model's predictions.

The published ReAct work studies how a language model can alternate reasoning and actions against an external environment. Reflexion studies feedback carried into later attempts through a linguistic memory. SWE-bench frames real GitHub issue resolution as an executable software task. Those are useful design precedents for bounded tool use, persistent lessons and issue-level assessment [5–7]. They do not supply a thermodynamic validity test. NeqSim's physical engine, source version and unit-aware results must remain visible independently of the agent's reasoning trace. The NIST digital twin program likewise emphasizes defining requirements, validating and maintaining models, and producing actionable recommendations [8]. This book's living task structure is an implementation pattern for that recurring evidence problem, not an assertion that the public reference case satisfies every digital twin requirement.

A stronger evaluation program would compare three operating modes on the same archived tasks: an engineer's conventional periodic review, a deterministic NeqSim monitor without agent triage, and the same monitor with triggered agent review. Hold source data and model version fixed. Measure time to a correct diagnosis, false decisions, missed changes, review effort and reproducibility, not simply the number of generated reports. For software campaigns, compare issue resolution and post-merge regression rates over repeated runs, including tasks that remain blocked. Such studies require careful sampling and independent adjudication; the present book specifies the experiment rather than claiming its outcome.

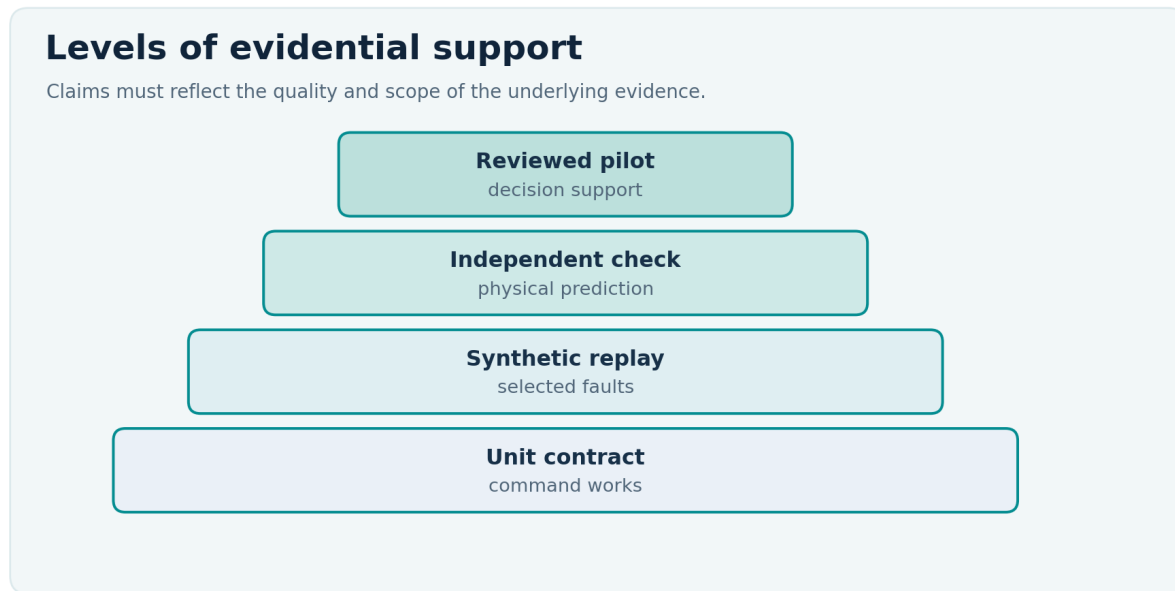


Figure 12.1: Claims should be phrased at the highest level actually supported by evidence.

Claim	Evidence required
The CLI command works	Command output and an inspectable cycle on the cited version.
A detector finds selected faults	Backtest with specified events, quiet periods, delays and false alarms.
A model predicts a physical KPI	Withheld or independent comparison over its stated domain, with uncertainty.
A candidate meets a goal	Validated objective, constraints, bounds and sensitivity checks.
A plant action produced value	Reviewed implementation record and subsequent measured outcome.
An agent can maintain a campaign	Repeated run evidence across merges, failures, reviews and changing requirements.

For a live digital twin, the state of the art is not a claim that all loops must be autonomous. It is a disciplined combination of model validation, data quality, versioned evidence, uncertainty, bounded agent work, human decision points and measurable backtests. NeqSim’s public living task implementation provides a concrete starting point. The breadth of its performance across facilities remains an empirical question, not a conclusion from one synthetic station.

12.1 Evaluation measures that matter

For monitoring: detection delay, false alarms per month, missed events, source availability, reproducibility and time to a useful diagnostic. For solving: feasible candidate rate, objective improvement, uncertainty, constraints, rounds and cost to reach a stop state. For coding campaigns: issues with reproducible evidence, tests passing on the exact head, review iterations, regressions after merge and time until a usable release. Report negative results and blocked runs; otherwise the apparent rate of progress is biased.

Chapter 13

Governance and deployment

Begin with a local public CSV task and no automatic agent. Backtest it. Add a task specific model stage with independent checks. Introduce read only external data through a documented adapter. Schedule monitoring only after the initial task basis, source contract and alert handling are reviewed. Enable a bounded agent for exception triage once its permissions, runtime and output are inspectable. A server deployment needs the same folder contract, pinned dependencies, a service identity, failure logs and a clear owner. These are steps to verify, not a claim of production qualification.



Figure 13.1: Reviewers examine the model, comparison and underlying evidence before accepting a new baseline. Conceptual illustration.



Figure 13.2: Increase autonomy as evidence, access control and review capacity mature.

The default advisory boundary is simple: the runner may compute and record, the agent may investigate and propose, and a named reviewer decides ledger status and baseline promotion. Any connection that can alter a physical process needs a separate control design, authorization and operating procedure. Keep read and write permissions distinct. Never include secrets or private source rows in public logs, figures or PRs. A public reproducible case should be generated from synthetic or lawfully sharable data.

If a source becomes stale, the correct result may be “no conclusion this cycle.” If a stage cannot reproduce its earlier result, stop promotion and investigate. A task should fail visibly when an output is zero, NaN, dimensionally wrong or a placeholder, rather than making the failure look like a valid KPI. These checks are part of the goal contract and software tests.

Chapter 14

Practical reference

14.1 Command map

Command	Role	Output to inspect
task-reference-case	Create a public synthetic learning case	New task folder and plan.
task-living	Scaffold continuous state for an existing task	Draft goal, baseline and ledger.
task-cycle	Run one monitor or solve cycle	cycles/<id>/ and living report.
task-solve	Repeat goal directed rounds until a stop state	Round cycles and stop reason.
task-backtest	Replay archived data without touching live state	Backtest report and event score.
task-schedule	Preview or install periodic monitoring	OS scheduler entry or printed cron line.
task-ledger	List or decide proposals	Append only event history.
task-promote	Move a reviewed cycle to the baseline	Archived baseline and reviewer note.
task-status	Inspect one or many living tasks	State, phase and next action.
task-report	Rebuild the living or formal report	LIVING_REPORT.md and optional formal output.

14.2 Terms

Baseline. The reviewed set of values and provenance used for comparison. **Cycle.** One time-stamped run with its source, KPI, trigger and diagnostic artifacts. **Goal.** A confirmed target and constraints with explicit stop and reopen rules. **Watermark.** The last source time safely consumed; it advances only after a suitable pull. **Trigger.** A recorded reason to inspect or reopen, not a diagnosis. **Ledger.** Append only events for proposed, accepted, implemented and verified improvements. **Promotion.** A reviewer's decision to make one cycle the new baseline. **Back-test.** Isolated replay of historical or synthetic evidence under a simulated clock. **Campaign.** A recurring issue to PR software improvement workflow with a persistent engineering goal.

14.3 Reproducibility record for this book

This book's implementation descriptions were checked against the public NeqSim default branch and docs/development/CONTINUOUS_TASK_SOLVING.md, devtools/neqsim_continuous/cli.py, the public skill and agent, AGENTS.md, and ImprovementCycle.java on 27 September 2026. The figures are original conceptual drawings generated for the book. The numeric compressor examples and timeline plot are explanatory and synthetic. The text does not assert that a fresh NeqSim

simulation or live facility backtest was executed in preparing this edition. The reader's first run of the reference case supplies its executable output.

14.4 Further reading

1. Solbraa, E. (2026). [Industrial Agentic Engineering with NeqSim](#). Foundations, physics, agents and skills.
2. Solbraa, E. (2026). [Agentic Engineering for Oil and Gas Facility Operations](#). Data, tools and multi specialist workflow.
3. NeqSim contributors. [Published Continuous Task Solving guide](#), [guide source](#), [living task skill](#), [CLI source](#), [Java improvement cycle](#), and [agent instructions](#). Source inspected September 2026.
4. NeqSim community. [Community skills](#), [continuous improvement toolkit](#), [community agents](#), and [continuous task improvement agent](#). Accessed September 2026.
5. Yao, S., et al. (2023). [ReAct: Synergizing Reasoning and Acting in Language Models](#). Research on interleaved tool actions and reasoning.
6. Shinn, N., et al. (2023). [Reflexion: Language Agents with Verbal Reinforcement Learning](#). Research on feedback across attempts.
7. Jimenez, C. E., et al. (2024). [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#). Issue based software benchmark.
8. NIST. [Digital twins research program](#). Model requirements, validation, maintenance and actionable recommendations.
9. Montgomery, D. C. (2019). Introduction to Statistical Quality Control, 8th ed. Wiley. Sequential monitoring background.
10. Jones, D. R., Schonlau, M., and Welch, W. J. (1998). Efficient Global Optimization of Expensive Black Box Functions. *Journal of Global Optimization* 13, 455–492. Expected improvement background.
11. Evensen, G. (2009). Data Assimilation: The Ensemble Kalman Filter, 2nd ed. Springer. Ensemble parameter estimation background.